Current routing and operating boundaries

Intake: Power Apps freezes the envelope and UUID. The flow validates, conditionally appends, verifies the immutable GitLab file, and responds. Only a successful new-save response triggers the owner Teams notice.

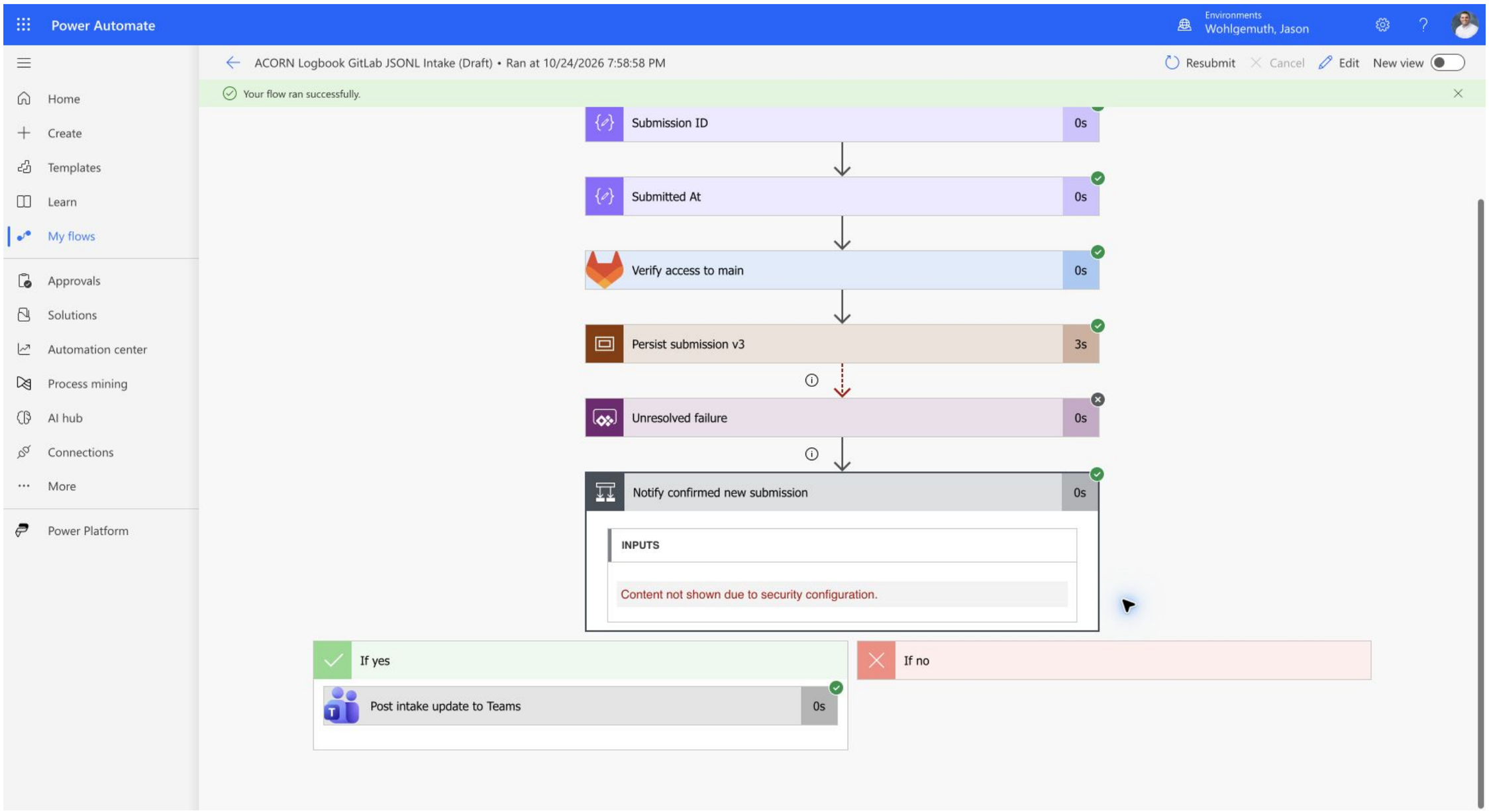
Callback: the owner-only manual flow validates an event, claims or finds its Dataverse receipt, compares the exact payload, and sends once through its frozen destination. Sent replays skip delivery; uncertain Sending stays held for reconciliation.

Teams: mr-created and report-no-changes. **Email:** review-needed, artifact-ready, and failed. Existing receipts retain their original destination, including earlier routine email receipts.

Existing connections: Microsoft Teams Default-52fd4, Office 365 Outlook Default-29833, and the protected GitLab connections. The app flow binding was refreshed, saved, published, and exercised. Intake checker warnings requested that refresh; there were no checker errors. External callback authentication and Vergil delivery remain unimplemented.

CURRENT WORKFLOW / DEVELOPER

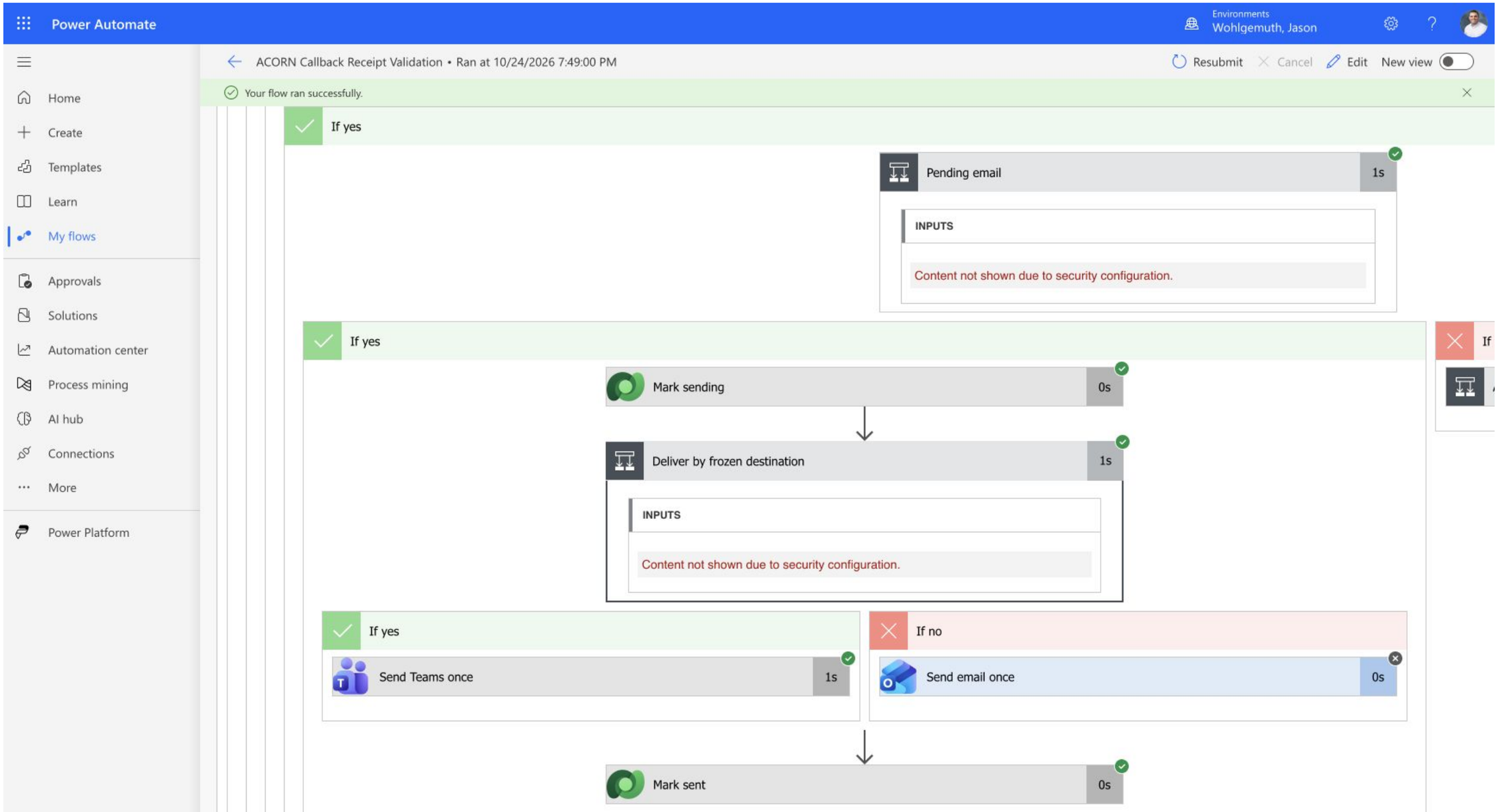
Confirmed capture precedes the Teams notice



Run 08584087205468981614810029297CU09 succeeded. The unresolved-response action skipped; the Teams action succeeded. A notice failure cannot undo the already confirmed GitLab write.

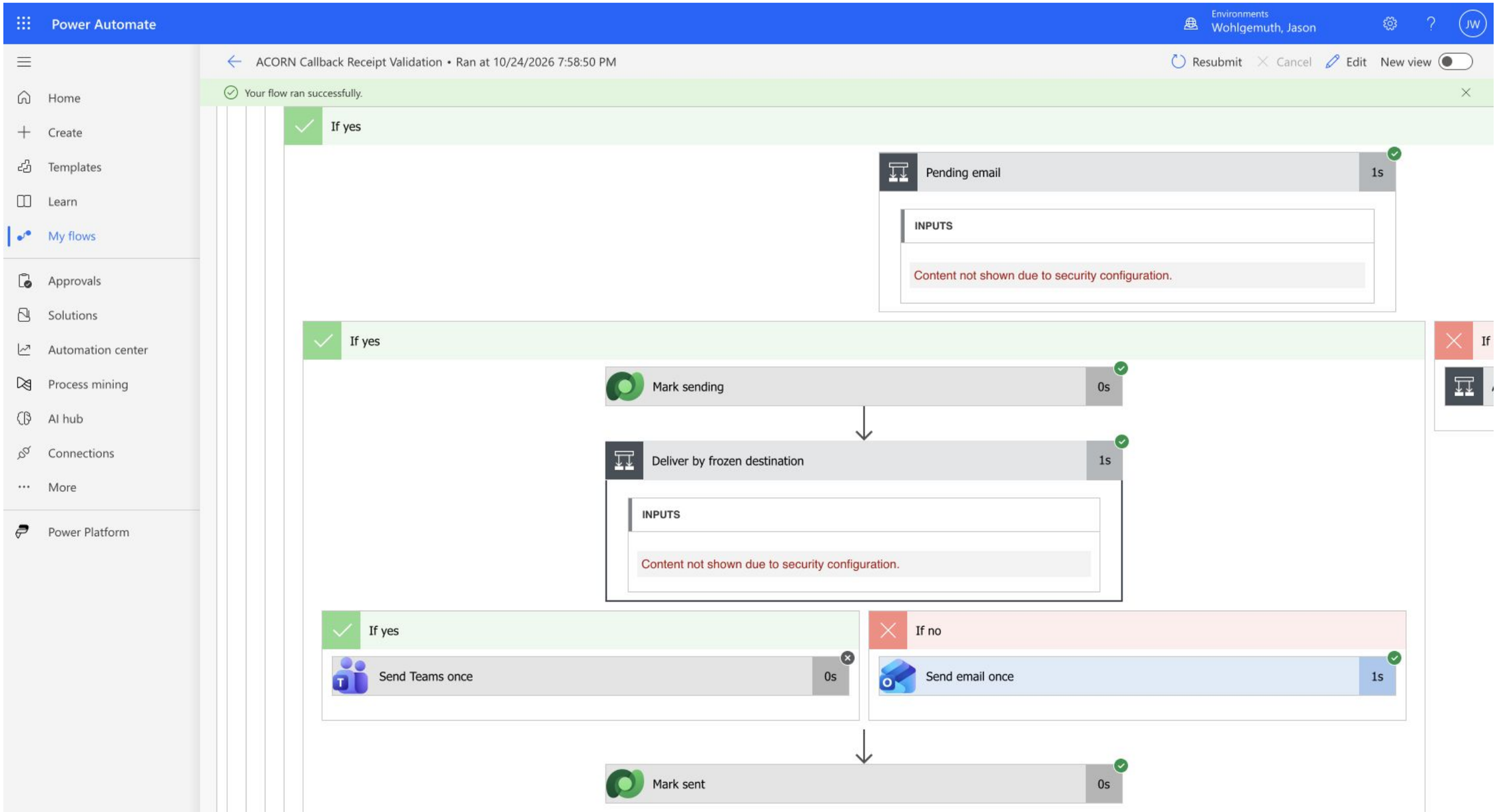
CURRENT WORKFLOW / DEVELOPER

Routine callback goes to Teams



The report-no-changes test succeeded in Teams and skipped Outlook. Its exact replay reached Duplicate no send. The receipt ledger remains authoritative for callback delivery state.

Milestone callback keeps email



A labeled failed-event test used Outlook successfully and skipped Teams: run 08584087205548109223433666322CU07. Inbox receipt was not independently inspected.

Run the manual milestone and recover safely

1. Open ACORN Callback Receipt Validation in Wohlgemuth, Jason. Choose Run. Enter a stable Webhook ID and exact Event JSON matching the schema. Use the actual event after the stage occurs; never represent a synthetic fixture as real review or publication.

2. Confirm the run and receipt state. Exact replay uses the same ID and identical JSON. Do not create a new ID to work around an uncertain delivery. Inspect the original attempt and Teams chat or Outlook Sent Items before reconciling Sending.

3. Intake notices are best effort, with automatic send retries disabled. If a save succeeds but its notice fails, retain the GitLab receipt and reconcile the notice separately. Replaying intake does not send another saved-highlight notice.

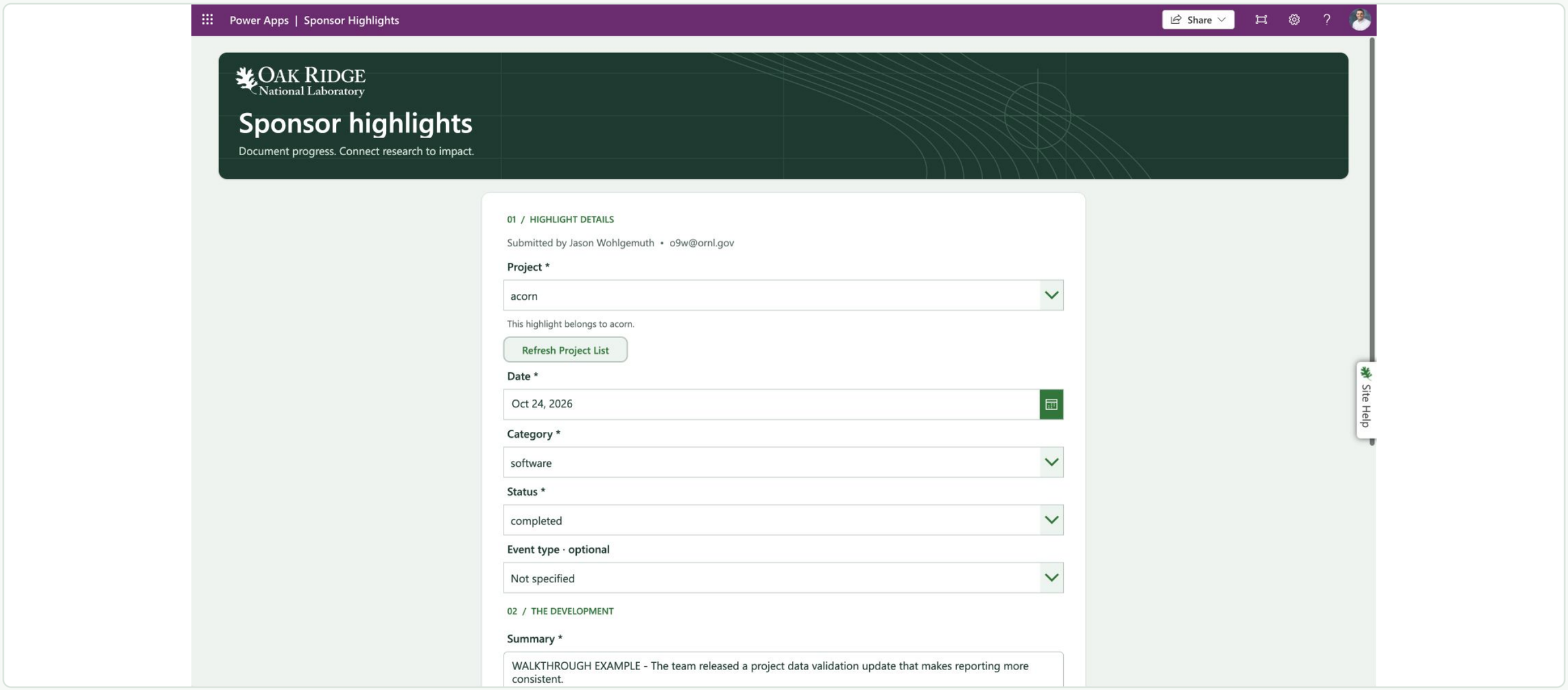
Legacy Dataverse column names still say email: emailrecipient stores the frozen route (teams-self:o9w@ornl.gov or email), and emailstate/emailattemptrun track the single selected channel. Group-channel delivery needs an approved destination and separate channel state in PA-12. The source bundle is not an importable solution.

Sponsor Highlights: developer guide

A screenshot walkthrough for developers and flow owners: inspect catalog and JSONL intake runs, reconcile immutable receipts, operate the manual callback receiver, and understand remaining acceptance work.

1. Current pilot: Wohlgemuth, Jason. Operational owner and initial notification recipient: Jason Wohlgemuth (o9w@ornl.gov).

2. This guide uses the published form and controlled test evidence from October 24, 2026. Examples are synthetic and excluded from sponsor reporting.



Current published form. The sample entered for these screenshots was not submitted.

Keep in mind: Saving a highlight queues it in the logbook. ACORN admission, human review, report generation, and a report notification are later steps.

Open Sponsor Highlights

What happens, and who does it

There are three Power Automate flows. Contributors normally use only Sponsor Highlights; owners use the flow histories and the receipt ledger.

Stage	What it does	Pilot status
Project Catalog	Loads valid project folder choices from the associated repository.	Enabled; used by the app.
JSONL Intake	Validates a frozen submission, appends one logbook line, and returns a commit receipt.	Enabled; fresh append and exact replay verified.
ACORN / Vergil and human review	Admits the queued record, prepares review work, and later produces reports.	Separate integration work; saving in the app is not evidence these steps ran.
Callback Receipt Validation	Claims an event ID, tracks its email outcome, and notifies Jason.	Owner-only manual receiver tested; external callback authentication is not configured.

Keep in mind: The intended path is Form > GitLab logbook > ACORN admission > human review > report > notification. The pilot currently proves the form/intake path and the manual notification receiver independently.

Using this guide

Use the user and reviewer guide for form entry and content review. This edition covers the Power Automate implementation and operational checks.

Topic	Pages	Purpose
Architecture	2	Understand the flow boundaries and current pilot state.
Intake and catalog	4-8	Inspect runs, configuration, commits, replay, and failures.
Callback operations	9-14	Validate events, inspect durable receipts, and reconcile email delivery.
Reference	15-17	Interpret outcomes, review remaining gates, and locate evidence.

Keep in mind: The app and flows are in Wohlgemuth, Jason. Captions identify earlier test screenshots and current captures. No new flow run was performed for this split.

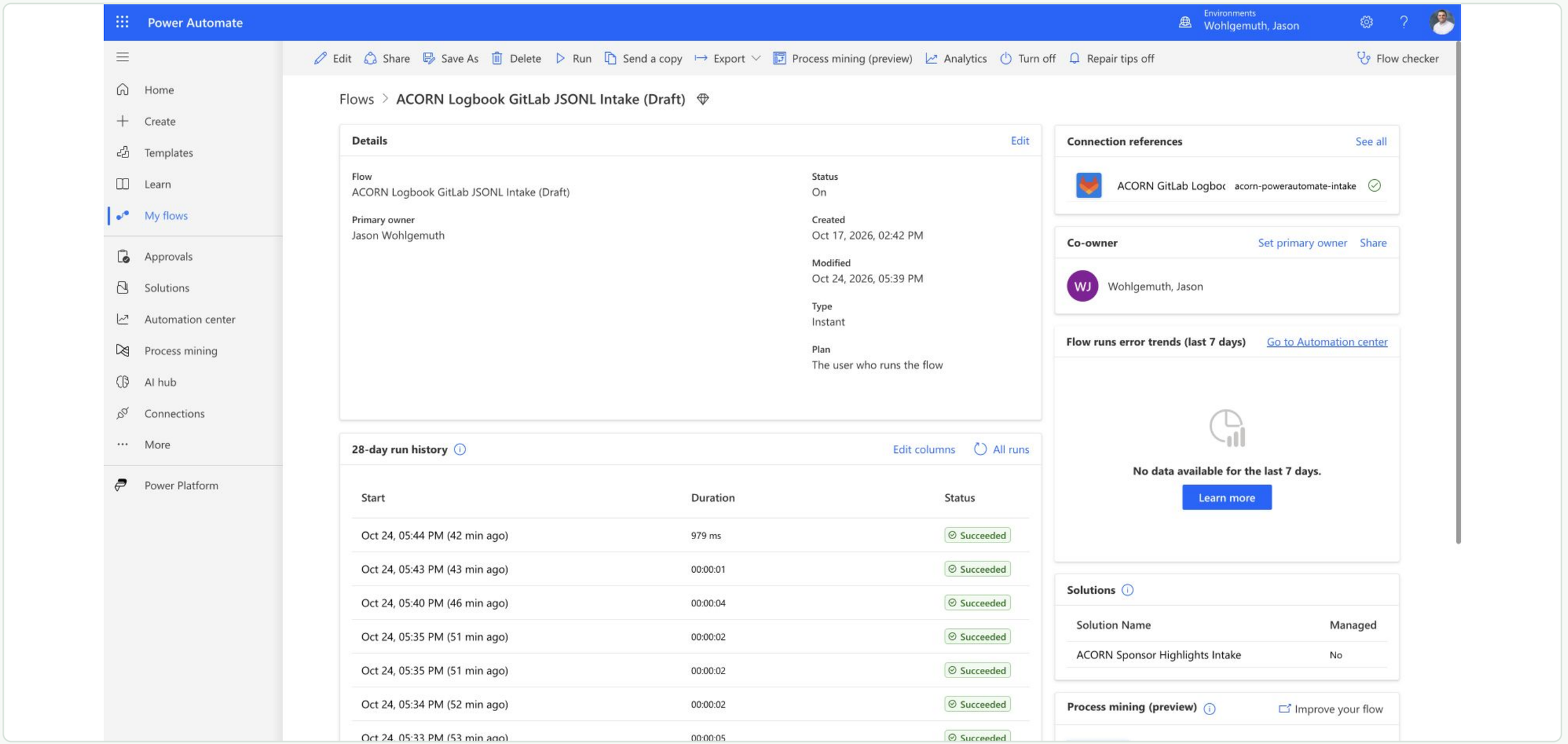
Find the intake flow and a run

Open Power Automate in Wohlgemuth, Jason, then ACORN Logbook GitLab JSONL Intake (Draft). The retained Draft name does not mean the flow is disabled.

1. Confirm Status is On. Match the run time to the contributor's receipt or reported attempt.

2. Open the relevant entry in run history and expand the action that first failed or returned a disposition.

3. A green overall run is insufficient by itself: confirm the response branch and immutable GitLab evidence.



Current capture: enabled intake flow, owner, connections, and run history.

Open intake flow

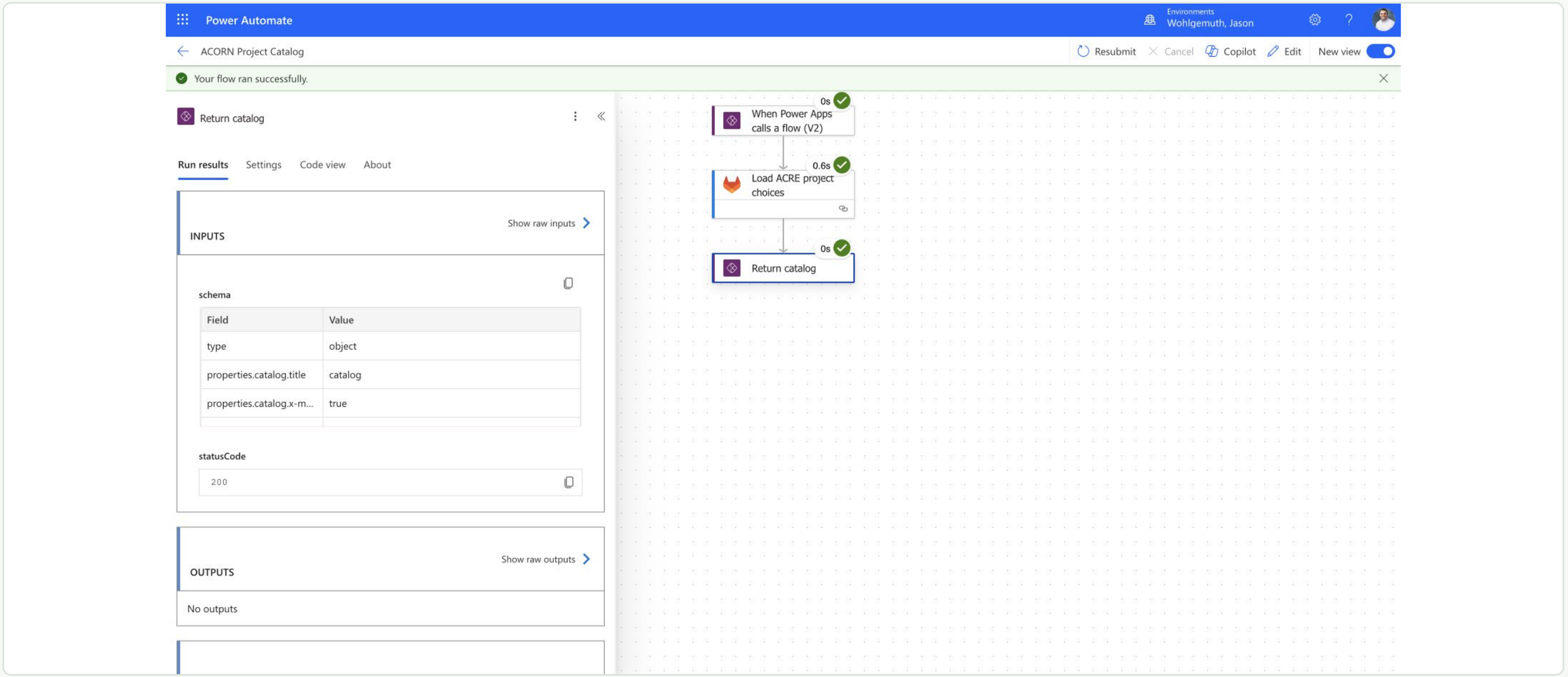
Inspect project-catalog behavior

ACORN Project Catalog returns choices from one repository revision. The app calls it to populate Project.

1. If .acorn.jsonc has folders, use that list, including an explicitly empty list. Canonical IDs are the choices; aliases do not create duplicate options.

2. Without folders, discover parent folders containing index.json or index.yaml. Root indexes and symlinks are excluded.

3. Invalid configuration or denied access stops discovery. Inspect the actual run; do not treat an empty list as permission to invent a project.



Earlier verified catalog response: inspect projects, source, and revision in the return action.

Keep in mind: A new catalog load also succeeded while capturing this guide. This does not resolve the earlier intermittent write-access failures.

Open catalog flow

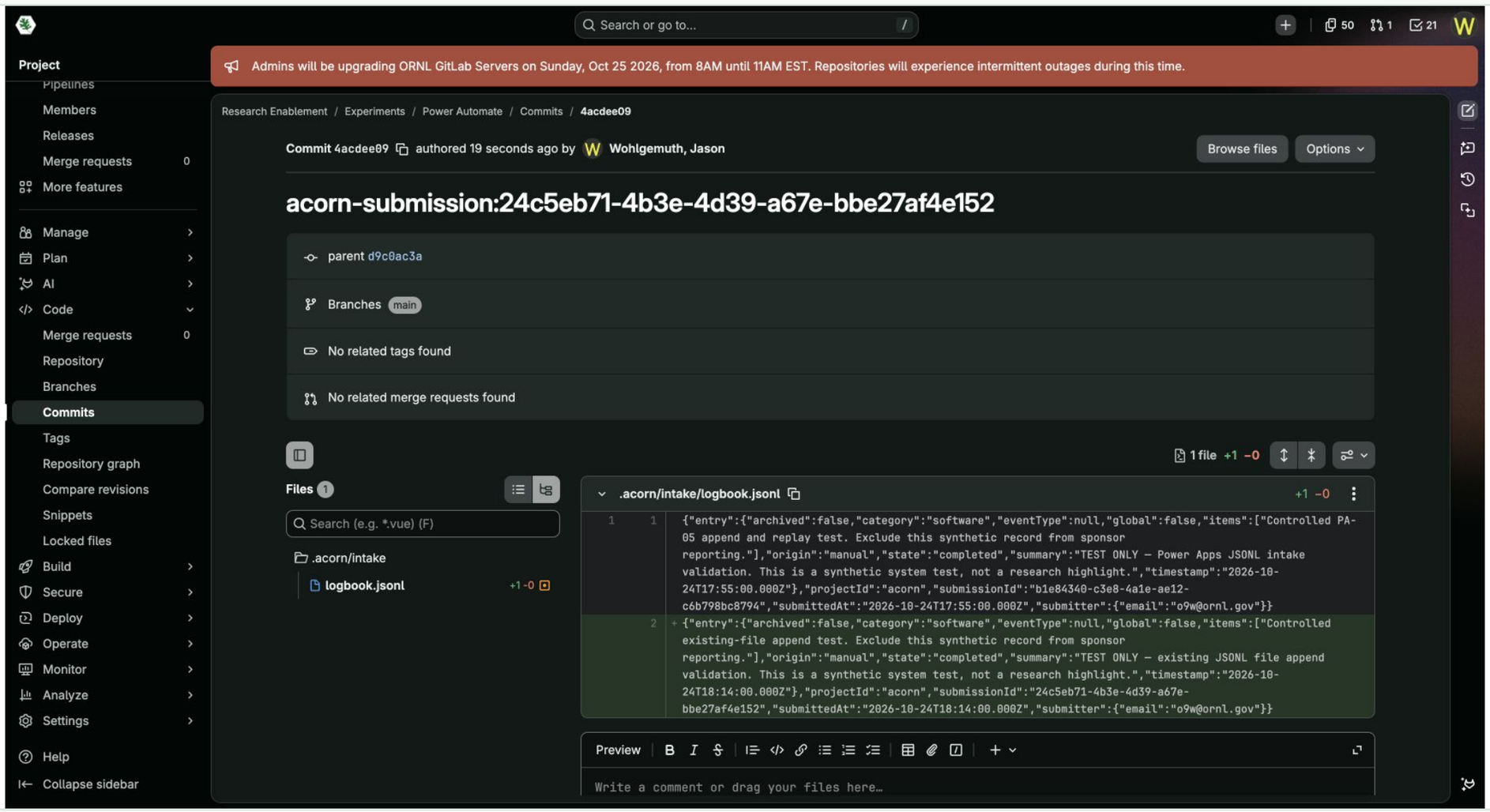
Follow a new intake submission

The intake flow validates the caller and envelope, checks immutable history for replay, then conditionally appends one line.

1. For new work, current catalog membership must match the selected project. Existing records must be preserved.

2. Commit append has automatic write retries disabled. Verify written snapshot must prove the immutable result before Queued new.

3. Check the JSONL file in project 22464 on main and compare the submission ID, project, caller, and fields with the frozen submission.



Earlier verified existing-file append: one added JSONL line, with the original line preserved.

Keep in mind: Destination: .acorn/intake/logbook.jsonl. Keep all TEST ONLY evidence out of sponsor reporting.

Open pilot repository

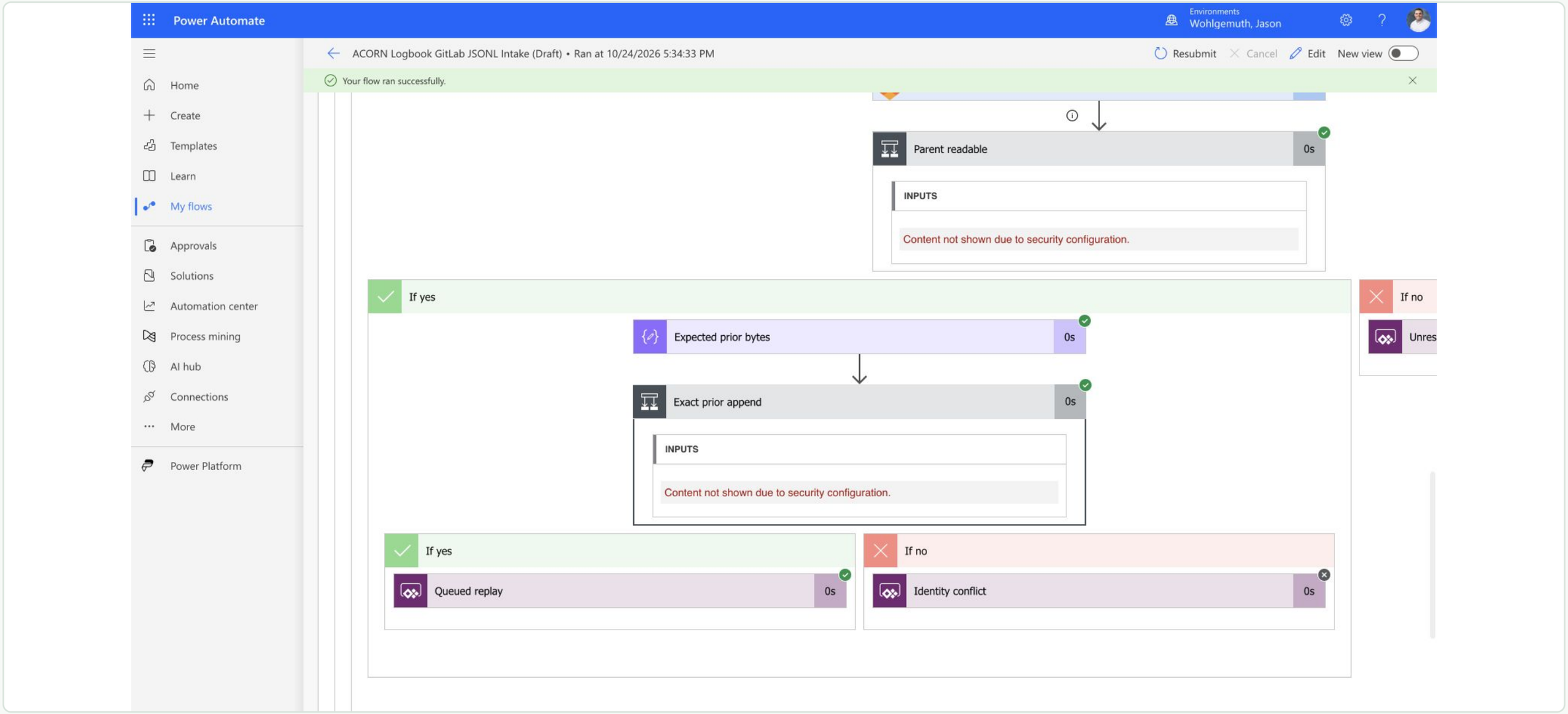
Recognize a successful replay

Retrying the exact original submission should reconcile an existing immutable commit and return Queued replay without another append.

1. Find prior commit and parent-snapshot comparison evidence, then Queued replay.

2. Confirm the new-submission branch was skipped and the receipt points to the original commit.

3. For ambiguous outcomes, preserve the same input. A history search bound or unavailable history may legitimately return unresolved.



Verified replay through the replacement connection: immutable comparison succeeded and Queued replay ran.

Keep in mind: The flow searches up to 100 file-history commits. If proof cannot be established, it must not claim success.

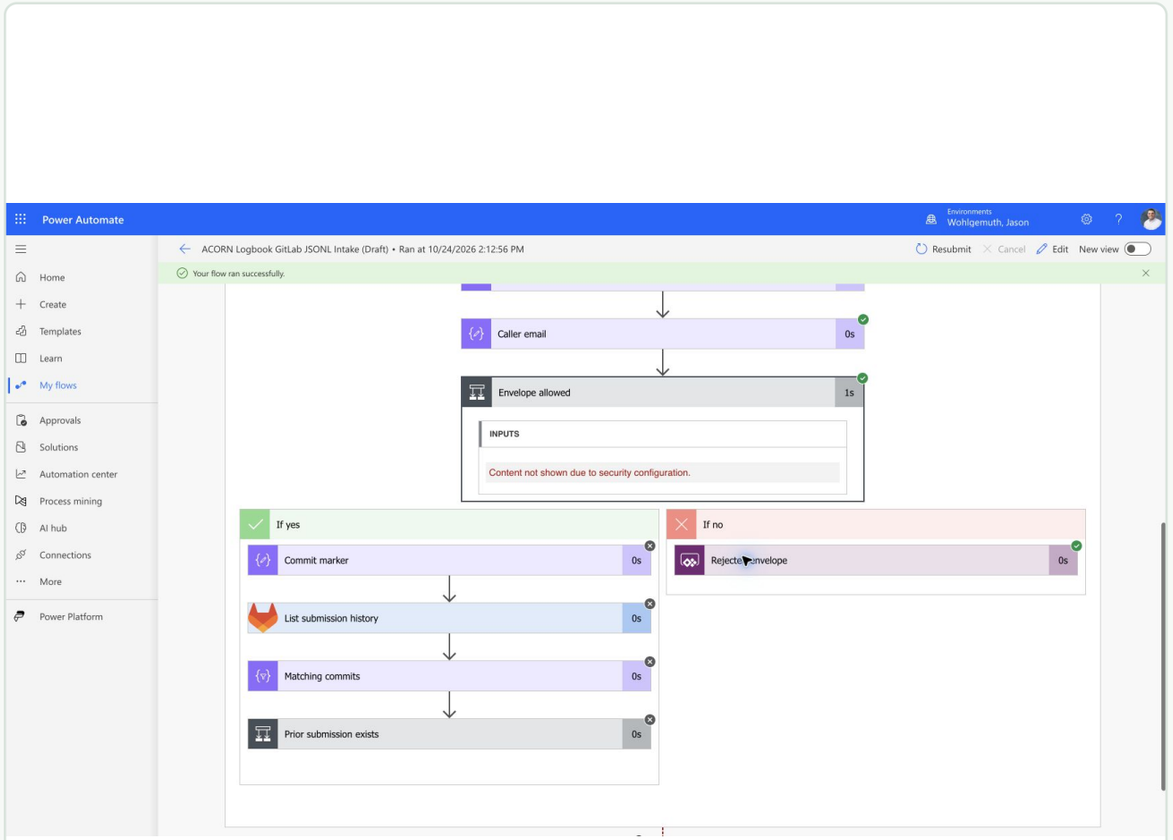
Investigate validation and access failures

Read the failed action and the response path; do not interpret every red run as a failed write.

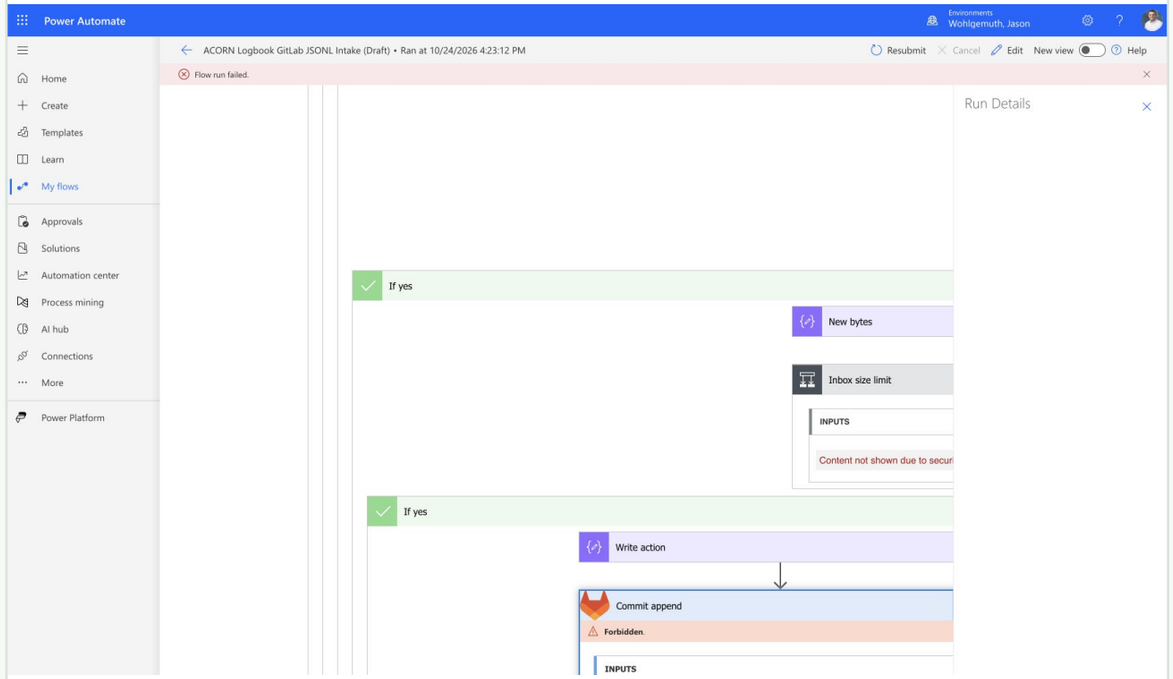
1. Invalid schema or caller: expect downstream write actions to be skipped. Isolated over-limit summary/detail tests passed without writing.

2. HTTP 403: preserve the run ID and time, and have the owner coordinate approved network/access investigation. Intervening successful calls do not prove the issue is resolved.

3. An unsuccessful or uncertain write must use the unresolved response. The duplicate-response repair is saved; its live failed-commit acceptance remains outstanding.



Earlier verified caller mismatch: Rejected envelope; no append.



Earlier explicit network-range 403 at Commit append. This evidence predates the single-fallback repair.

Keep in mind: Payload-bearing history may still echo validation-error values. The pilot is restricted to non-sensitive highlights.

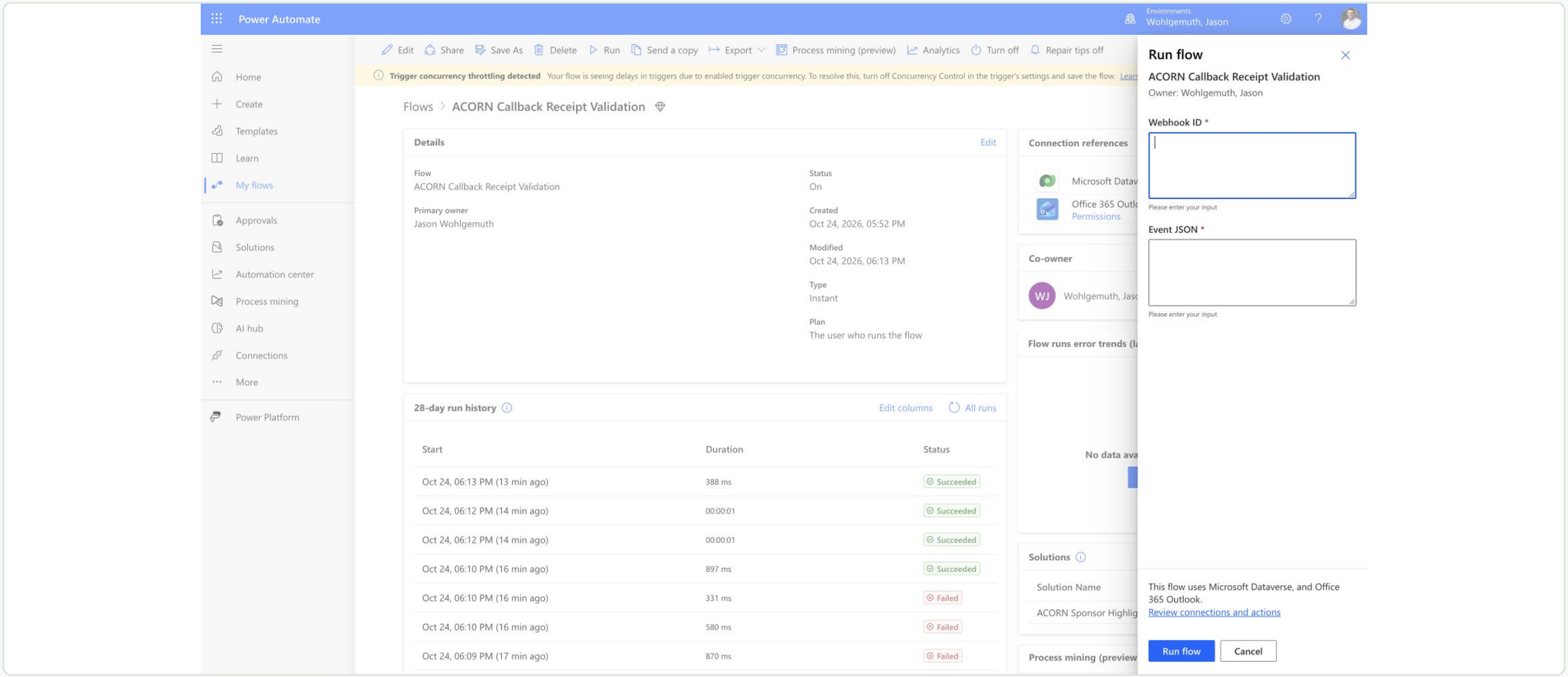
Run the manual callback receiver

ACORN Callback Receipt Validation is an owner-only test/recovery flow. It is not yet an externally callable bot endpoint.

1. Open the flow and choose Run. Supply the exact Webhook ID and Event JSON for the intended event.

2. For recovery, use the existing ID and byte-for-byte original JSON. For a new approved test, use a distinct clearly synthetic ID.

3. Run flow can send an email to the frozen recipient, currently Jason. Do not run arbitrary examples just to inspect the designer.



Current capture: both manual input fields. No run was submitted while creating this guide.

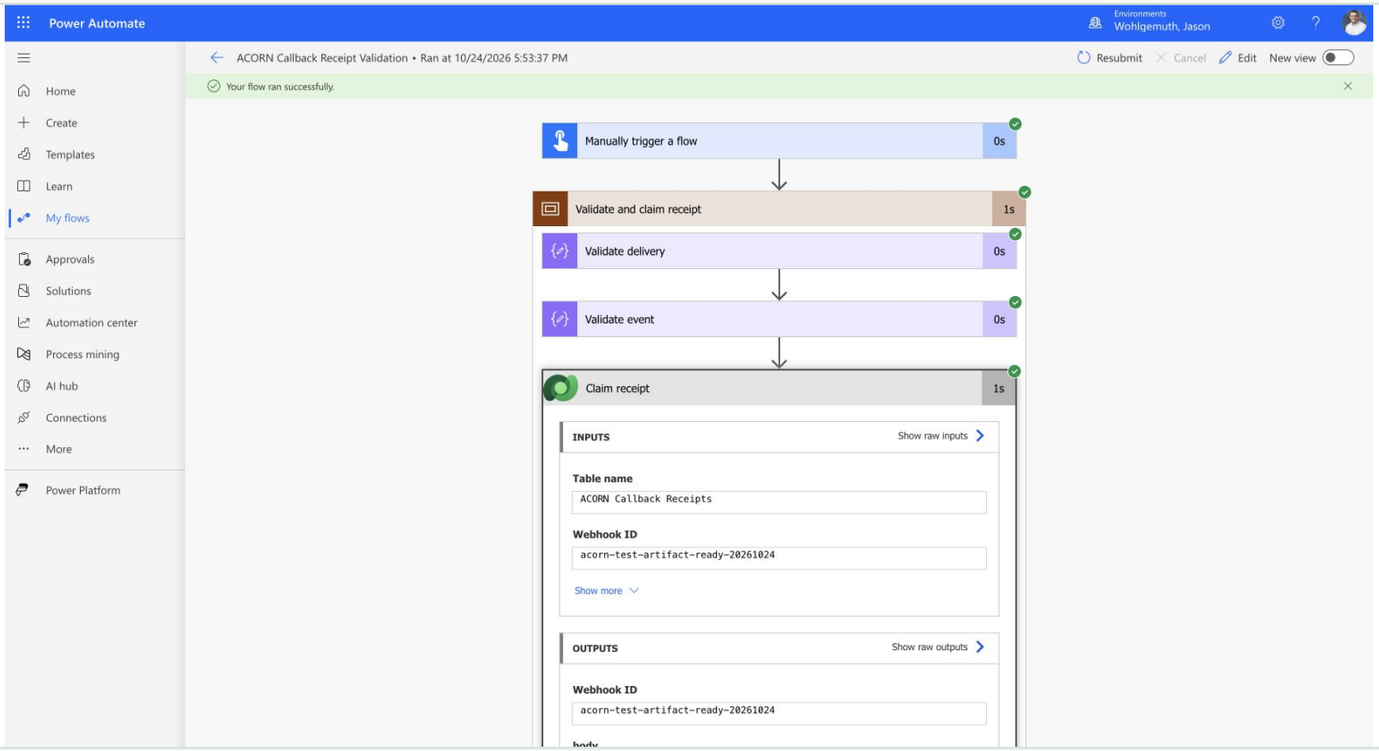
Keep in mind: The screenshot may show a concurrency warning. Concurrency one is intentional for notification serialization; do not disable it to remove the warning.

Open callback flow

Use the exact callback event shape

Each event is identified by event and uses snake_case field names. Unknown fields, missing fields, wrong types, and unknown events are rejected.

event	Other required fields
artifact-ready	checksum, package_url, project_id, through
failed	message, project_id
mr-created	merge_request_url, project_id
report-no-changes	project_id, through
review-needed	merge_request_url, project_id



Earlier live claim: the event was validated and a durable receipt created.

Keep in mind: Do not add a submission UUID or operation ID to these event bodies. The manual Webhook ID input stands in for the eventual webhook-id HTTP header. All five event fixtures and 36 offline contract checks passed.

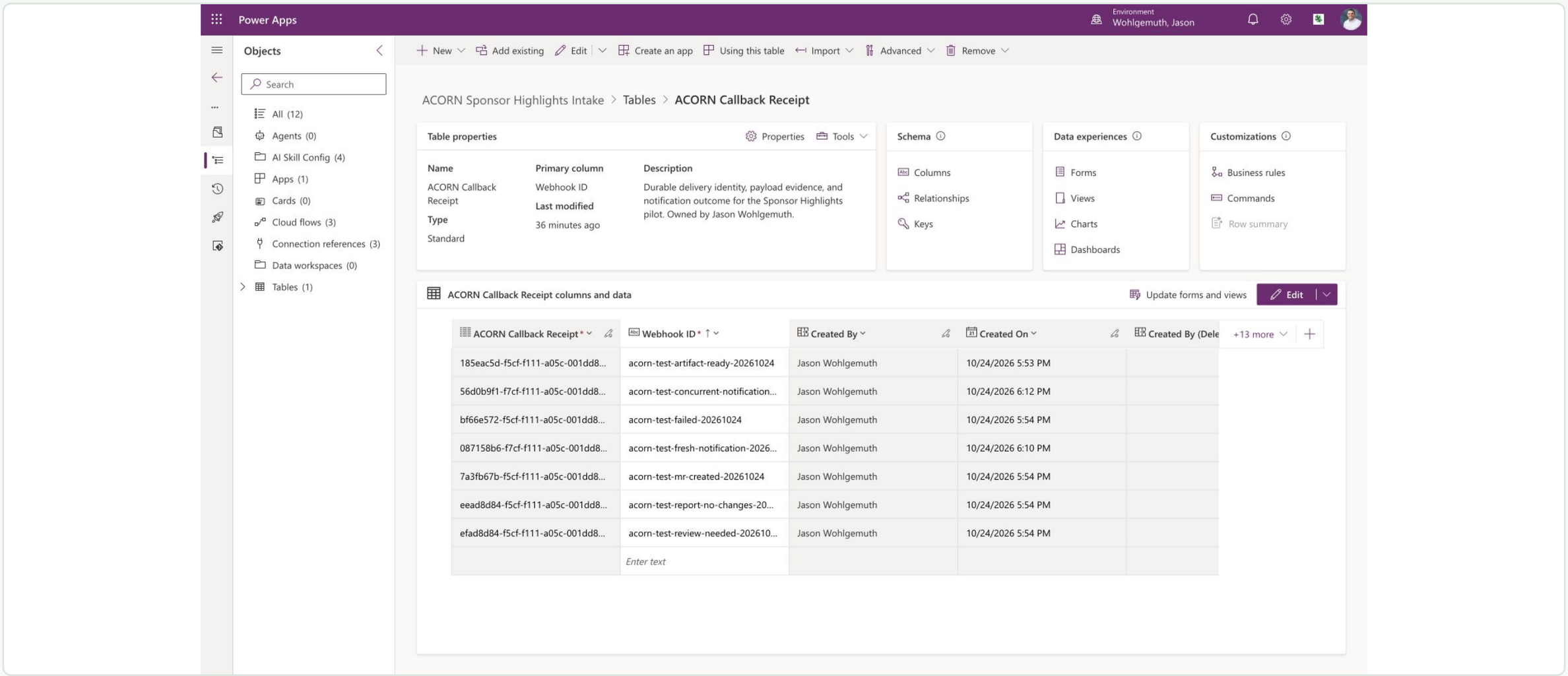
Read the durable receipt ledger

Open ACORN Callback Receipt in the pilot solution. Webhook ID has an active unique key, so two claims cannot create two receipts with the same ID.

1. Find the exact Webhook ID. Inspect Payload JSON, Email Recipient, Email State, and Email Attempt Run; scroll horizontally to those columns when needed.

2. Payload and recipient are frozen at claim time. A changed payload under an existing ID is a conflict.

3. Pending means channel work may proceed; Sending means an attempt has been recorded; Sent is the recorded completed send.



Current capture: seven synthetic receipt rows. Delivery columns are to the right of the initially visible columns.

Keep in mind: These synthetic ledger rows are separate from the five TEST ONLY JSONL inbox records. Do not treat either set as real reporting input.

Open callback receipt table

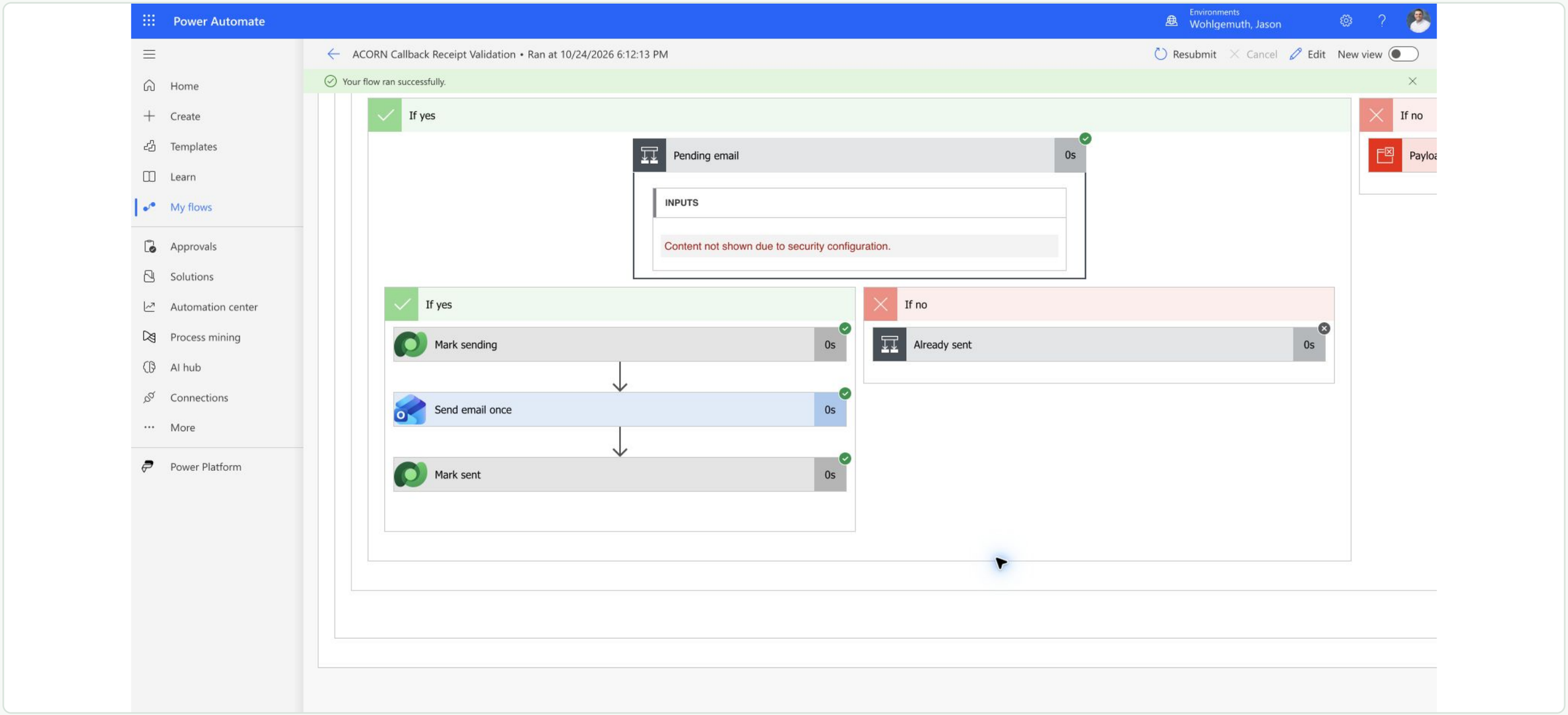
Verify an email send

The receiver records the attempt before invoking Outlook, then marks the receipt Sent only after the send action succeeds.

1. Inspect Mark sending > Send email once > Mark sent. All three should succeed for a new send.

2. Compare the stored Email Attempt Run with that run. Outlook acceptance is verified; independent inbox receipt was not inspected.

3. The pilot sent three controlled TEST ONLY emails. Future destination changes affect new claims; existing receipts keep the original recipient.



Verified flow: all three sending actions succeeded. Content is hidden by security settings.

Keep in mind: A future Teams group channel needs its own action and delivery state. It is part of the shared-environment migration work.

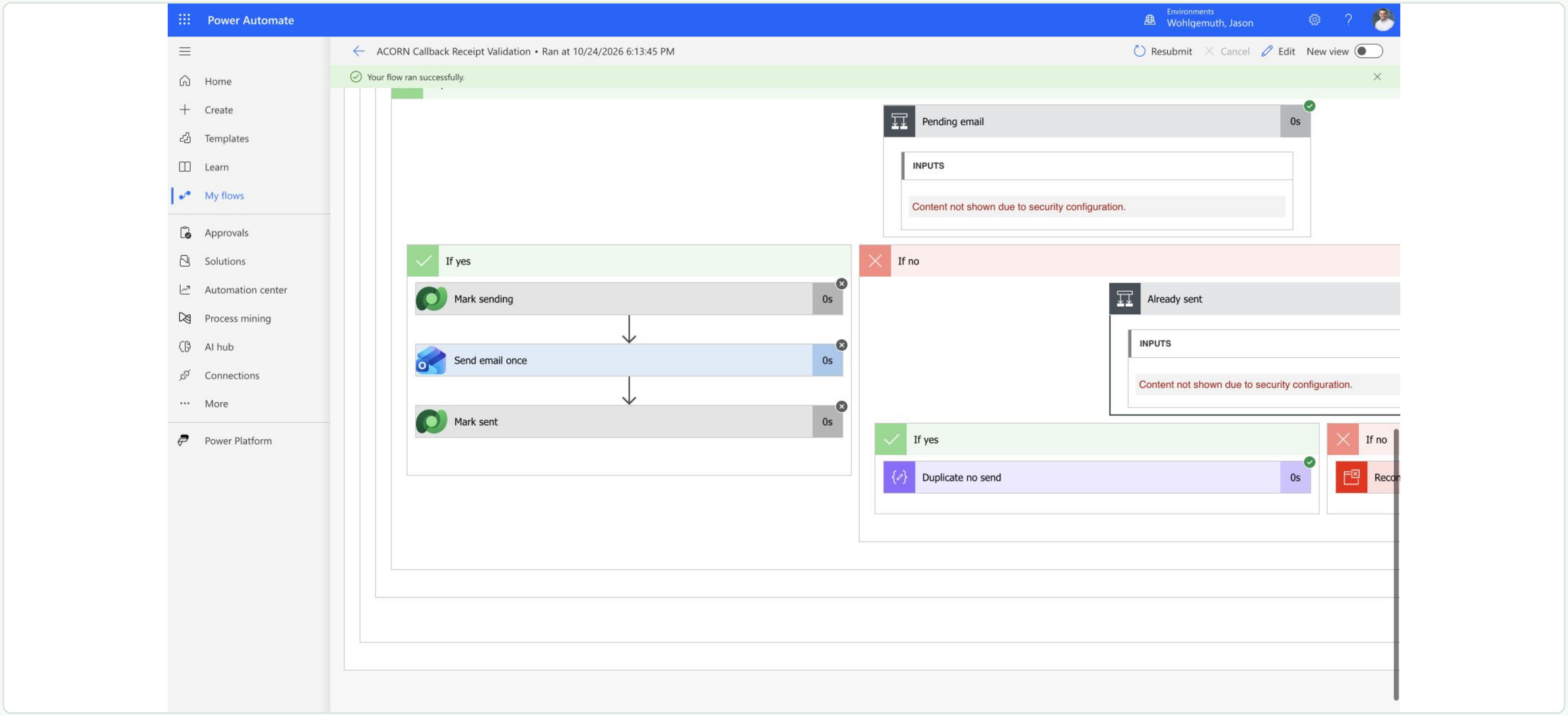
Confirm duplicates do not notify again

A duplicate claim may appear as a failed action because the unique key rejects a second row. Inspect the delivery scope before deciding the whole operation failed.

1. For the exact payload on a Sent row, expect Duplicate no send to succeed and the sending actions to be skipped.

2. Sequential replay, two simultaneous identical requests, and replay after disabling/re-enabling the flow all preserved a single email effect in the tests.

3. Keep trigger concurrency at one. A second independent receiver would need an additional atomic per-channel delivery claim.



Existing acceptance run after flow reactivation: Duplicate no send, with sending actions skipped.

Keep in mind: A repeated ID with changed payload is rejected instead of taking this duplicate-success branch.

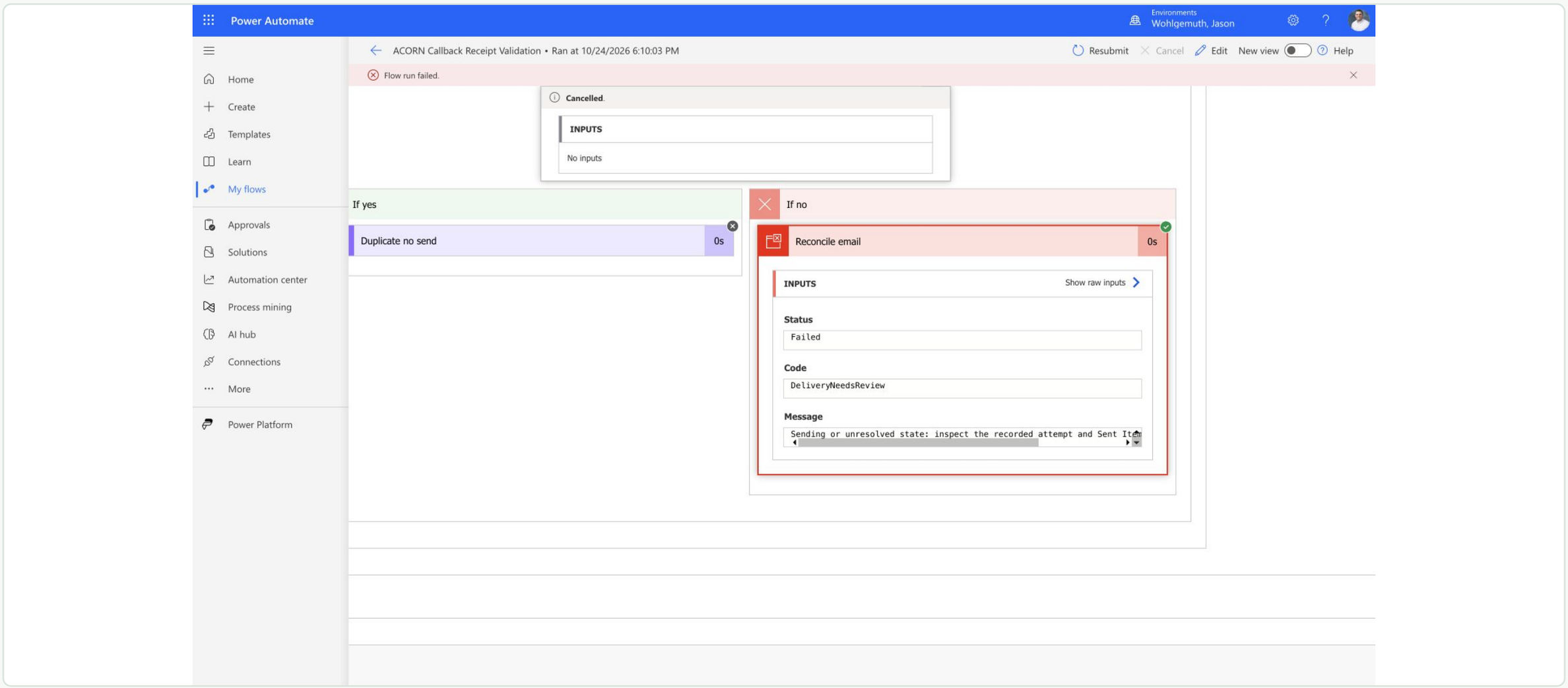
Hold uncertain email for review

Sending or another unresolved state must stop at Reconcile email with DeliveryNeedsReview. It is not safe to assume a timed-out connector sent nothing.

1. Inspect the recorded attempt and the sending account's Sent Items before any resend.

2. If delivery is proven, reconcile the receipt to Sent and retain the attempt ID. If non-delivery is proven, document the investigation before authorizing an identical-payload retry.

3. If delivery is still ambiguous, leave the record held. Do not reset it to Pending merely to get a green run.



Verified guard test: Reconcile email ended the run with DeliveryNeedsReview. The row was a marked simulation; no email was attempted for it.

Keep in mind: This verifies the hold branch, not a real connector timeout after possible delivery. A non-idempotent email connector cannot guarantee exactly one effect across every ambiguous failure.

Result and action reference

Use the displayed outcome to choose the next step. A new submission and a retry are not interchangeable.

What you see	What it means	Next action
Submit highlight unavailable	Required fields or a safety check are not satisfied.	Complete required fields; read counters and status.
Loading projects...	Catalog request is running.	Wait; do not repeatedly click.
Highlight saved	Matching ID and commit receipt confirmed.	Back to highlight; then New highlight for distinct work.
Highlight needs a correction	Explicit rejection.	Back to highlight, then Edit highlight if offered.
Receipt needs review	Same ID has different stored content.	Keep the page and contact the owner.
Your highlight needs attention / Retry submission	Save could not be confirmed, or another safety check failed.	Read the message. Preserve pending content and retry the same ID when appropriate.
Browser recovery unavailable	Pending storage could not be prepared or restored.	Use the published regular-browser app; preserve existing pending data.
Duplicate no send	Exact callback already sent.	No resend; keep the original receipt.
DeliveryNeedsReview	Email attempt may be uncertain.	Owner reconciles attempt and Sent Items.

Keep in mind: The current app panel titles come from the saved formulas. Not every panel branch has a published visual acceptance screenshot; use the captioned server-side evidence for those branches.

Pilot boundaries and next steps

Use the current pilot for non-sensitive intake within its tested limits. These remaining items are tracked in the PA wiki plan.

Area	Current boundary / remaining work
Access reliability	Intermittent GitLab 403s still need approved access investigation. A successful catalog read during this guide does not close that gate.
Intake acceptance	Exact-limit, malformed-tail/size, concurrent writers/creators, and the repaired failed-commit path still need live acceptance.
Browser recovery	Reload and app-tab closure passed. Full process restart and published-browser storage-denial tests remain; use one app tab per pending slot.
Callbacks	Manual receipt/email behavior passed. External authentication, HTTP acknowledgement, and sender lost-response/restart integration remain.
Deployment	Pilot stays in Wohlgemuth, Jason. PA-12 covers an approved shared environment and group/channel destination.
Backup	The source bundle is available. A custom-connector inclusion error still prevents a complete importable solution export.

Keep in mind: No credentials or permission settings were changed to produce this guide. No new highlight or notification was sent during screenshot collection.

Links, evidence, and support

For a support request, include the receipt or Webhook ID, approximate time and timezone, selected project, visible message, and run ID if available. Do not send tokens or API keys.

1. Contributor evidence: confirmed commits 2618af3f..., 904ff8d6..., and 7dd33b6d...; recovery and immutable replay were verified.

2. Callback evidence: send run 0858408727239135312614250820 0CU12; simultaneous duplicate pair ends CU28/CU25; reactivation replay 085840872685974033189910836 35CU22.

3. Local evidence source: .wiki/assets/sponsor-highlights/README.md, saved formulas, screenshots, and acceptance transcripts. The wiki contains remaining work; the runbook contains completed evidence.

4. Use PDF bookmarks to find a topic and zoom into screenshots for individual action details. The companion user and reviewer PDF covers form entry, recovery, and the review handoff.

Keep in mind: Prepared October 24, 2026, America/New_York. Screenshots show local browser times; runbook timestamps marked Z are UTC. Revisit the PA plan before treating this as a shared production deployment.

Open Sponsor Highlights | Intake flow | Project Catalog flow | Callback Receipt Validation | Callback receipt ledger | Pilot GitLab repository

2:15-4:15 / LIVE LOCAL CLI

Reviewer: inspect, then approve

Stage and inspect are read-only with respect to the destination index. Confirm the project, authorized member, exact source hash and proposed one-entry diff.

```
acorn --offline intake stage --batch --config intake.json --inbox selected.jsonl
acorn --offline intake inspect --config intake.json --batch-id <staged-id>

canApply: true    accepted: 1    rejected: 0
project: acorn    source UUID: 4313fcfc-0201-441e-8bda-c77718a31dcf
```

Pause for the reviewer. After approval, apply the staged batch. Repeat the same apply to show that there is still only one entry.

```
acorn --offline intake apply --config intake.json --batch-id <staged-id>
status: complete    project status: applied

Repeated apply: destination bytes unchanged
```

Evidence: 01-stage.txt, 02-inspect.txt, 03-apply.txt and 04-apply-replay.txt. The full commands and digests are retained in rehearsal-evidence.json.

4:15-5:30 / LIVE LOCAL CLI

Reviewer: explicitly promote the contributor

The personnel summary produces one supported contributor candidate. Other highlight categories may have no graduation candidate.

```
acorn --offline intake candidates --index acorn/index.json

target: contributors
value: Example, Avery (TEST ONLY)
entry: form-4313-fcfc-0201-441-e-8-bda-c-77718-a-31-dcf
baseDigest: sha256:75d3f85f9de5a1c84a61d2e36d42d823...
```

After reviewing the candidate and source, select that entry and use the complete returned digest. The command validates the result and preserves a byte-exact backup.

```
acorn --offline intake graduate --index acorn/index.json \
  --accept form-4313-fcfc-0201-441-e-8-bda-c-77718-a-31-dcf \
  --expected-digest <complete-reviewed-digest>

+ contributor: Example, Avery (TEST ONLY)
~ selected source entry: archived = true
```

The original entry remains. Unrelated metadata is unchanged. Repeating the stale approval fails without adding another contributor.

6:30-7:30 / LOCAL ACCEPTANCE

Prove the safe repeat and rejection paths

Repeat ingestion

Index bytes remain unchanged; one logbook entry.

Repeat graduation

Stale digest is rejected; one contributor remains.

Unknown project

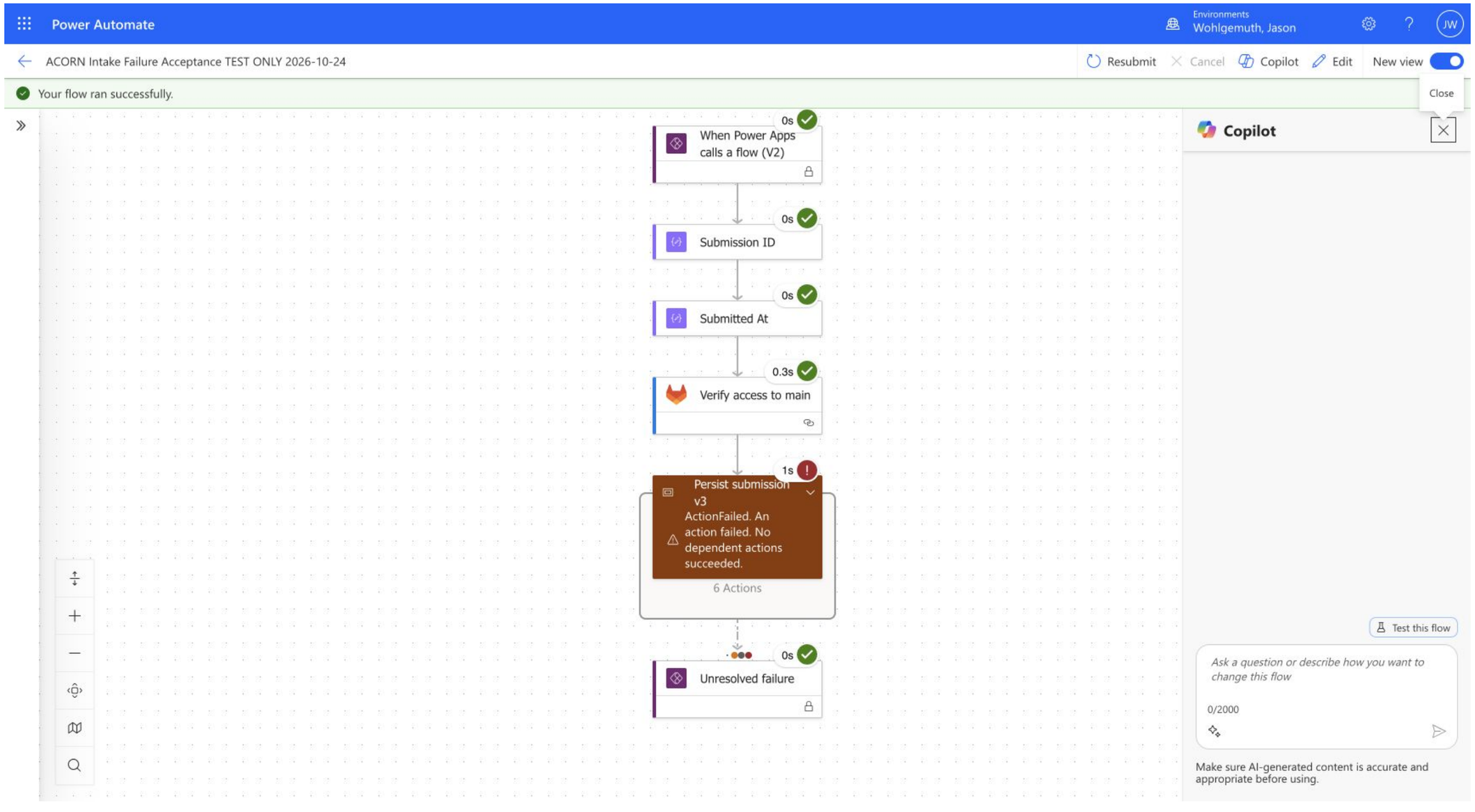
Stage marks the line rejected; apply refuses the batch.

Repeat reporting

Both renders leave the reviewed index unchanged.

All eleven automated rehearsal steps passed. The 2.01-second automated runtime excludes human review and presentation time.

A failed commit produces one unresolved response



A copied flow used a simulated Failed result for Commit append. Readback, success confirmation and the nested readback response were skipped; the single outer unresolved response succeeded. The test copy is disabled.

This proves the response graph, not a real GitLab timeout after a possibly committed write. Live intake was unchanged. Earlier guide screenshots and five-record counts predate the sixth demo submission.

Run the demo in a new disposable directory

Build the current CLI, then run this from the repository root. The workspace must not already exist. Two interactive approval checkpoints precede writes.

```
python3 .wiki/assets/sponsor-highlights/manual-demo/rehearse.py \
  --workspace /tmp/acorn-demo-next-week-01 \
  --acorn target/debug/acorn \
  --chrome '/Applications/Google Chrome.app/Contents/MacOS/Google Chrome'
```

Open the generated report/report.pdf and evidence.json. Use a new workspace name for each run. Keep the exact captured selected.jsonl; never regenerate its UUID or reformat it to bypass admission.

If cloud access is denied, say: "Recorded cloud capture, live local review and reporting." Continue with the saved receipt and immutable snapshot.

Future work: reliable approved egress/licensing, broader intake acceptance, shared deployment and manual-to-bot handback. Do not drain the remote inbox or start Vergil as part of this demo.